Uso abanzado de clases: Sistema de Autenticación de Usuarios

---

1. Objetivo de la práctica

Desarrollar una aplicación por consola que simule el acceso a una aplicación mediante:

- Registro de usuarios
- Inicio de sesión (login)
- Control de intentos fallidos
- Gestión en memoria
- Separación por capas de responsabilidad
- Validaciones mediante `RegEx`
- Uso correcto de Herencia y Objetos
- Uso de colecciones `(List, Set)`
- Manejo de excepciones
- Diseño limpio, modular y documentación de la solución

---

2. Arquitectura que vamos a implementar (separación por capas)

Debes de crear esta estructura a través de la creación de `paquetes` y `clases` que se indican.

```
src/
 ├── com.docencia.app/
 │    └── Main.java
 ├── com.docencia.model/
 │    └── Usuario.java
 │    └── Persona.java
 ├── com.docencia.service/
 │    ├── IAuthService.java
 │    └── IUserService.java
 ├── com.docencia.service.impl/
 │    ├── AuthServiceImpl.java
 │    └── UserServiceImpl.java
 ├── com.docencia.repository/
 │    ├── IUserRepository.java
 ├── com.docencia.repository.impl/
 │    ├── UserRepositoryImpl.java
```

```
        ├── com.docencia.util/
        │    └── Validaciones.java
```

- `Main`.- Menú de la aplicación.
- `Usuario`.- Modelo de datos de la aplicación extendido de `Persona`.
- `Persona`.- Modelo de datos básico, clase Abstracta que `NO se puede instanciar`.
- `IUserService`.- Interfaz de servicio que define las funciones de negocio sobre los usuarios `(Casos de Uso)`. Añadir, modificar o cosas de ese tipo. Se denomina en programación un **CRUD** (Leer `(R)`, Actualizar `(U)`, Añadir `(C)`, y Borrar `(D)`). Se realiza la implementación en la clase `UserRepositoryImpl` a través de `implements IUserRepository`.
- `IAuthService`.- Interfaz de servicio que define las funciones de negocio sobre el acceso de los usuarios `(Casos de Uso)`. Contiene las funciones que permiten deperminar si un usuario puede acceder/esta bloqueado, etc. Se realiza la implementación en la clase `AuthServiceImpl` a través de `implements IAuthService`.
- `IUserRepository`.- Interfaz de servicio que define las funciones de negocio sobre el acceso de los usuarios `(Casos de Uso)` que se encuentran en meoria. Esto es un `CRUD` básico, con las operaciones que hemos visto sobre `List/Set/Map`, básicamente `.add` \ `delete` etc.
- `Validaciones`.- Clase `static` que contiene la lógica para realizar las validaciones que se necesiten. Poor ejemplo para el `email`, validarEmail, para el `password`, validarPassword.

---

3. Responsabilidad por capa

model

Representa los datos y reglas internas del objeto.

- Pesona
- Usuario
- Validaciones básicas del constructor
- Normalización de datos

service

Contiene la lógica de negocio.

- Registro
- Login
- Bloqueo tras intentos fallidos
- Gestión de sesión

repository

Encargado del almacenamiento en memoria.

- Guardar usuarios
- Buscar por email
- Evitar duplicados

util

Métodos reutilizables:

- Validación email
- Validación contraseña
- Normalización

app

Interfaz por consola (menú).
NO contiene lógica de negocio.

👤 4. Modelo: Persona/Usuario

Atributos mínimos

```
private final int id
private String nombre;
private final String email;
private String password;
private int intentosFallidos;
private boolean bloqueado;
private final LocalDate fechaRegistro;
```

> Coloca las campos que consideres oportunos en la clase `Persona`, y aquellos que consideres oportunos en la clase `Usuario`. Reflexiona en que clase se utiliza el `id` para `equals/hasCode` y en cual `email`.

Reglas obligatorias

- nombre mínimo 5 caracteres
- email válido (RegEx)
- contraseña mínimo 6 caracteres
- fechaRegistro inmutable. `Se define cuando se crea el usuario`.
- email único en el sistema

🔐 5. Casos de uso obligatorios

1. Registrar usuario
2. Login
3. Listar usuarios
4. Buscar por email
5. Salir

🔄 6. Reglas de negocio importantes

Registro

- No permitir email duplicado
- Validar formato de email
- Validar contraseña

Login

- Si contraseña incorrecta → sumar intento
- Tras 3 intentos fallidos → bloquear usuario
- Usuario bloqueado no puede loguearse

🗄️ 7. Repository (almacenamiento en memoria)

Se recomienda usar:

```
private Set<Usuario> usuarios;
```

Clave: email normalizado.

Debe evitar duplicados automáticamente, o utilizar `List` que controle los elementos duplicados.

🧪 8. Validaciones (RegEx)

Clase: Validaciones

Métodos obligatorios:

```
public static boolean emailValido(String email)
public static boolean passwordValida(String password)
```

Objetivo de esta tarea

Debes de:

- Entender la diferencia entre modelo, servicio y repositorio
- Evitar lógica en el Main
- Comprender responsabilidades
- Aplicar encapsulación
- Diseñar código mantenible